Yanbu Industrial College

الهيئة الملكية للجبيل وينبع
Royal Commission for Jubail & Yanbu

Computer Science And Engineering Department

Academic Year 1447- 1448 / 2025- 2026

Second Semester

Project 1

CS381

Web Application Development

Date:	**3/5/2026**	**Time Allowed:**	0 0 Hours: 00 Minutes

Student Name 1: **Amjad Mohammed** **ID Number:**

Student Name 2: **Sara Mohammed** **ID Number:**

Teacher's Name: **Ms. Mai Nabeel** **Section:** **1**

Please read the following instructions carefully:

- Write your name, I.D. number and section number in the space provided above.
- Read each question carefully before answering.
- Answer all questions.
- Support materials are not allowed in the examination Hall except those provided by instructor.
- Do not use digital or printed dictionary.
- Do not use pencils for answering except for drawing.

For Instructor Use Only

CLO.	Mark Assigned	Mark Attained
1.3	5	
2.1	5	
3.1	5	
3.2	5	
	Total:	

Mr. Muhammad Rafiq **Signature:**

Marked by:

Dr. Mai Nabeel **Signature:**

Project Title: YIC Service Portal

Project Overview

You will build a **YIC Service Portal** over a period of **7 weeks** - a web application that serves Yanbu Industrial College students and staff. This is a **group project** with maximum of 2 students per group.

The project covers all course modules:

- **Frontend:** HTML5 (semantic), CSS (responsive), JavaScript (interactive)
- **Backend:** PHP (forms, sessions, file handling)
- **Database:** MySQL with PDO (CRUD operations)
- **Security:** Authentication, input validation, SQL injection prevention

Assigned Module

Form a group of 2 and select ONE primary service module listed below to implement in detail:

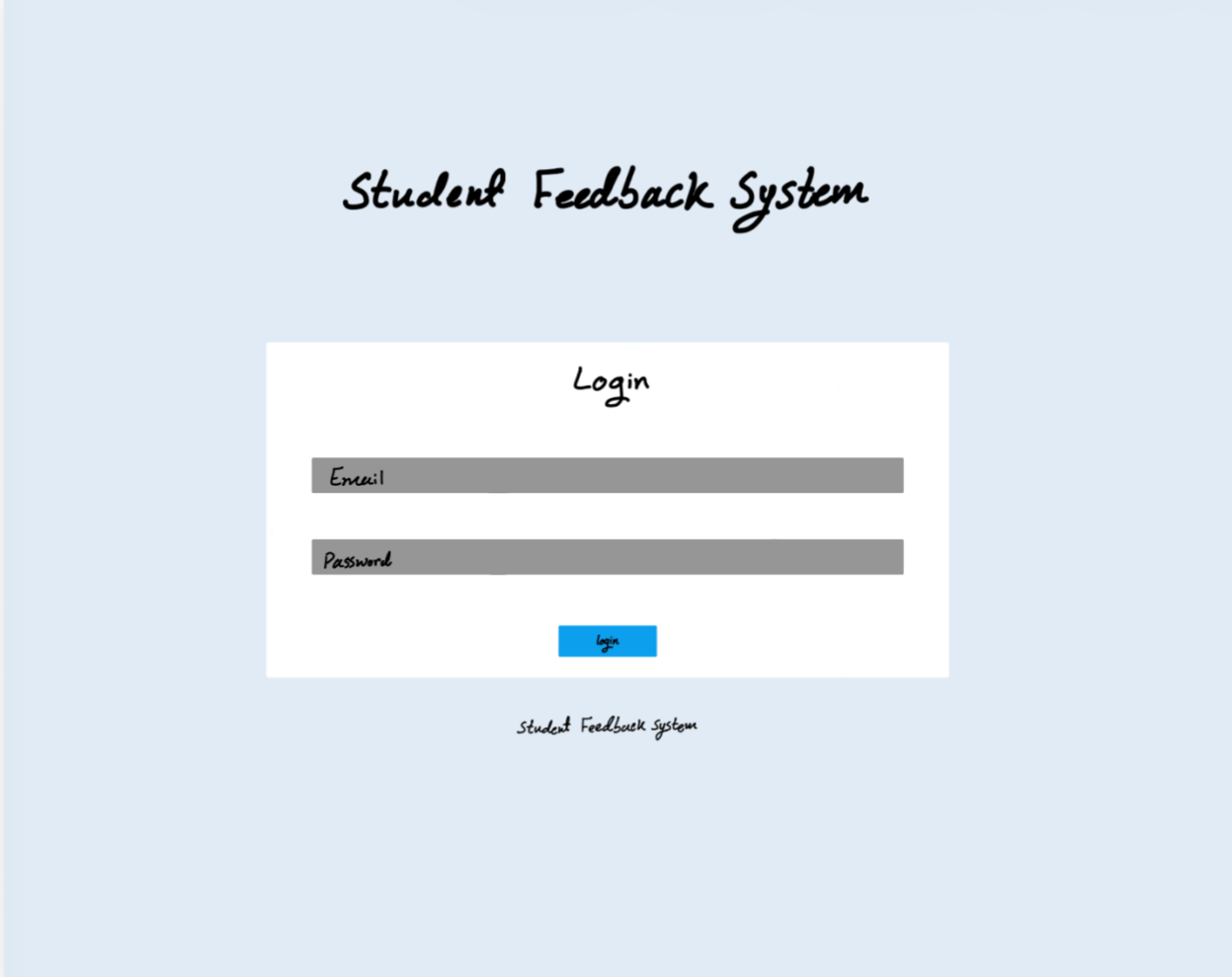
No	Primary Module	Description
1	Course Registration	Students browse courses, check prerequisites, register for sections, and view their semester schedule.
2	Library System	Search books, check availability, borrow/return items, view borrowing history, and track overdue fines.
3	IT Support Tickets	Users submit support requests, track ticket status, administrators assign technicians and resolve issues, and users provide feedback.
4	Event Management	View campus events, register for events, perform event check-in, and view an event calendar.
5	Lost & Found	Report lost items, post found items, manage claim requests, send notifications, and allow admin approval.
6	Room / Facility Booking	Students or staff reserve rooms (meeting rooms, labs, halls), view availability schedules, and manage booking requests and approvals.
7	Student Feedback System	Students submit feedback on courses or campus services, administrators review feedback, generate summaries, and track responses.
8	Campus Marketplace	Students can list items for sale (books, gadgets, etc.), browse listings, contact sellers, and manage their own postings.
9	Appointment Scheduling	Students book appointments with lecturers, advisors, or campus services; view available time slots and manage bookings.
10	Club & Society Management	Manage student clubs, membership registration, announcements, activity postings, and event participation tracking.

Project Phases & Deadlines

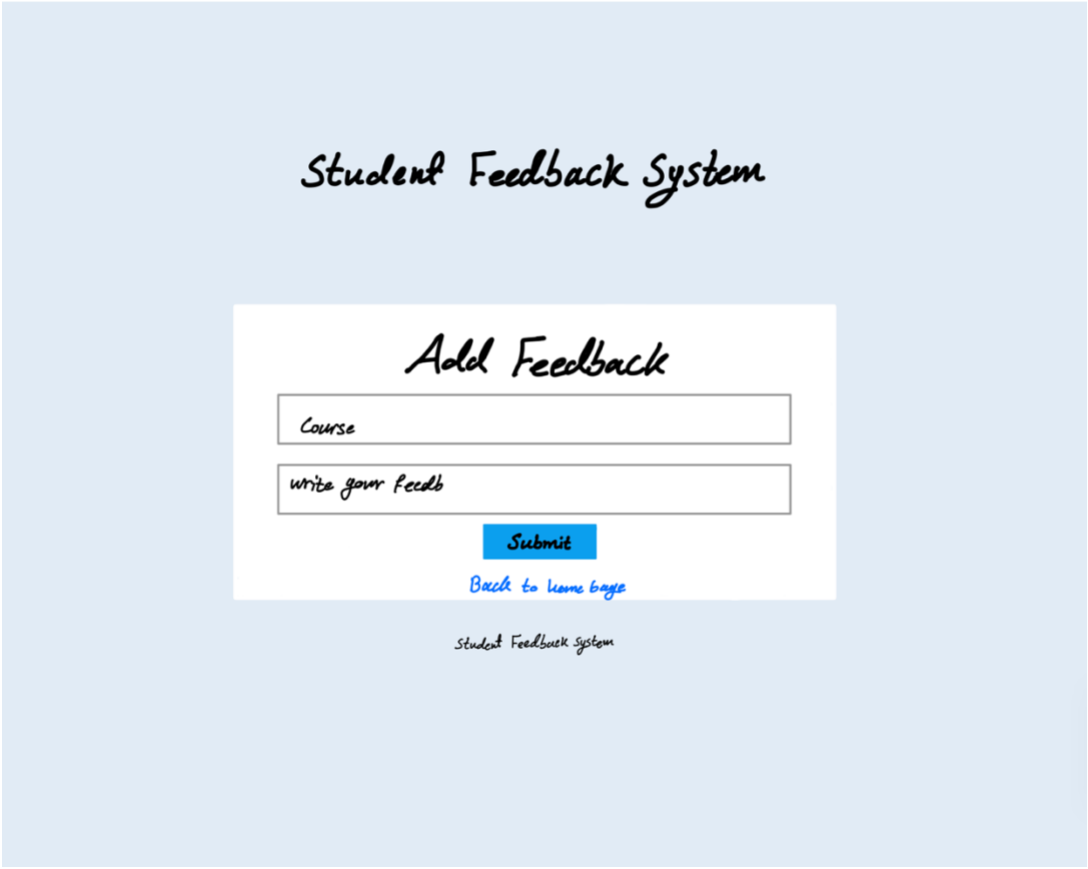
The project is divided into **4 phases** which must be properly documented in your final report.

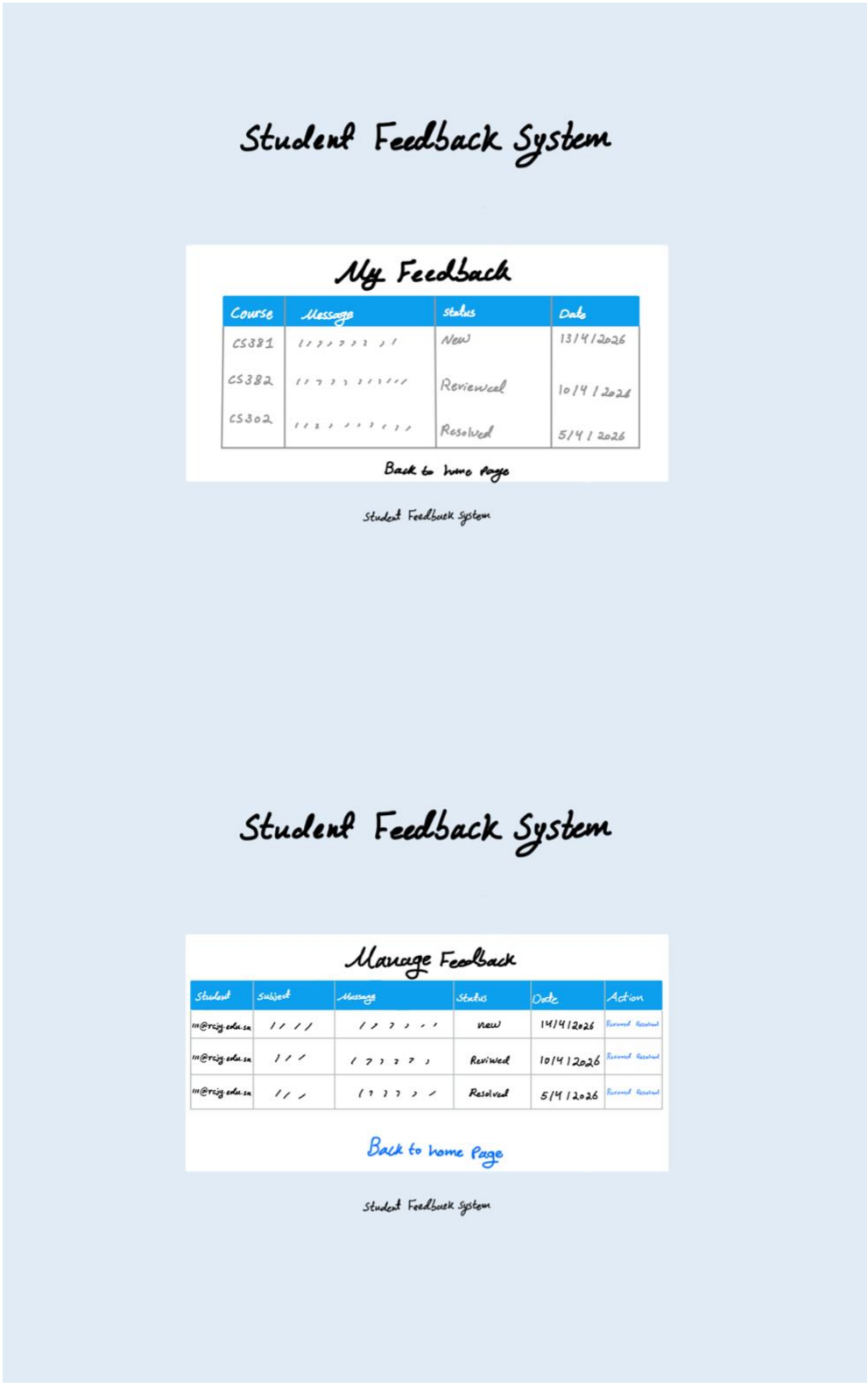
PHASE 1: Planning & Architecture Design

1. **Hand-drawn wireframes** (minimum 5 pages) - Photo/scan of paper sketches showing UI layout




Student Feedback System
Welcome, [Student Name]
Add Feedback
My Feedback
logout
Student Feedback System

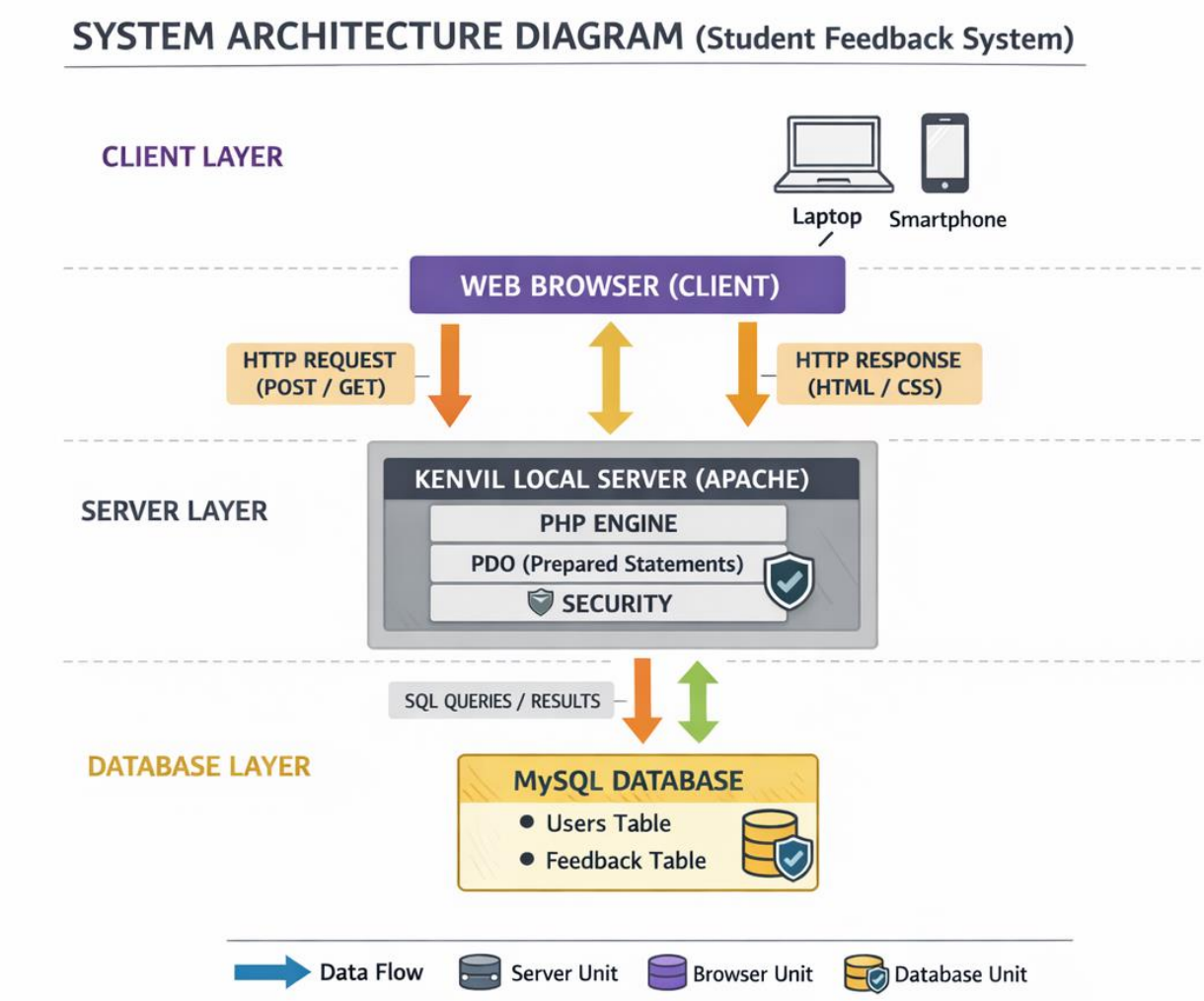

Student Feedback System
Add Feedback
Course
write your feedb
Submit
Back to Home page
Student Feedback System



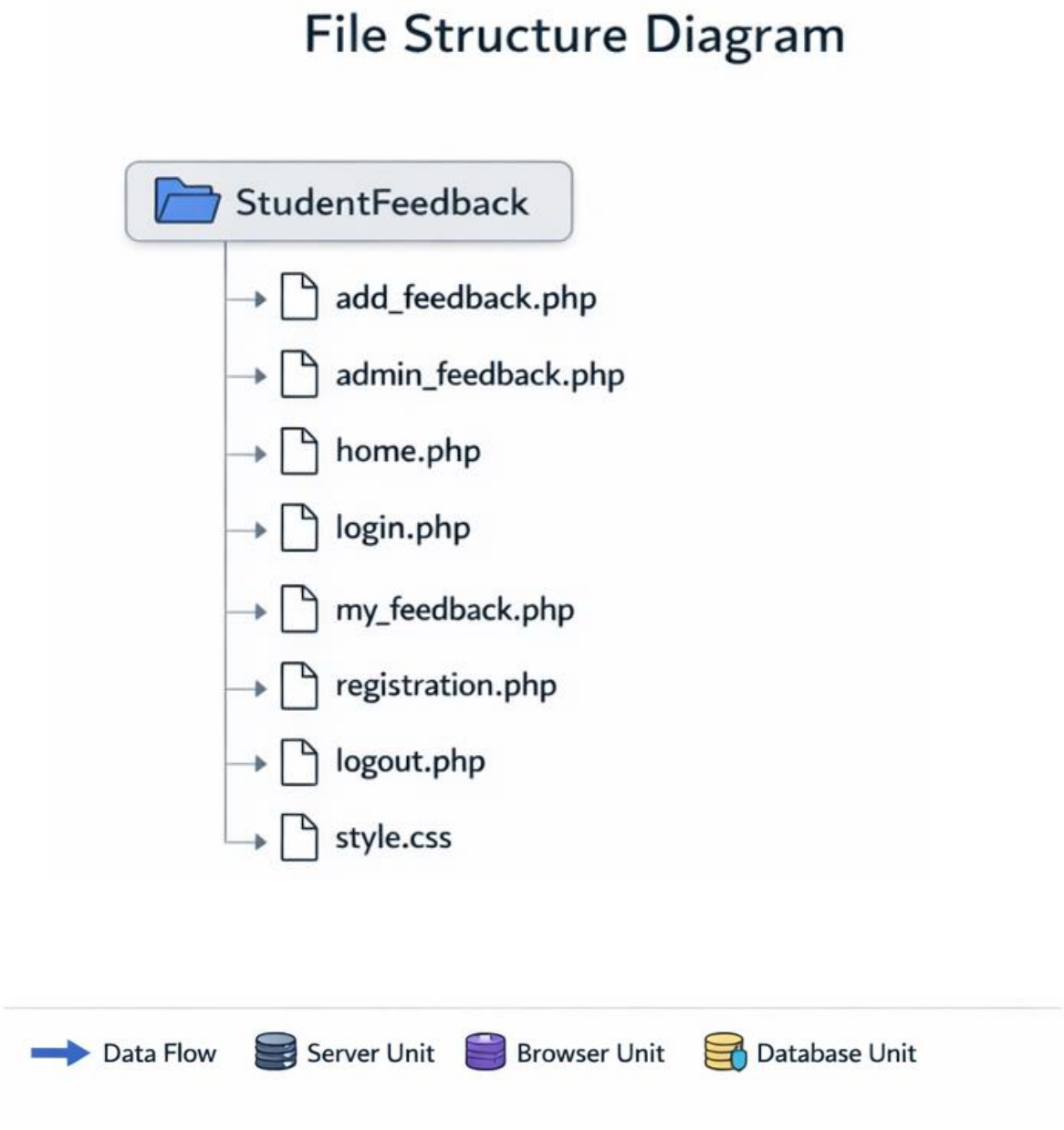
These wireframes present the core structure of the system's pages. They outline the user interface layout without including detailed design elements.

2. **System architecture diagram** - Shows Client → Server → Database layers and data flow

This diagram explains the system layers and data flow.

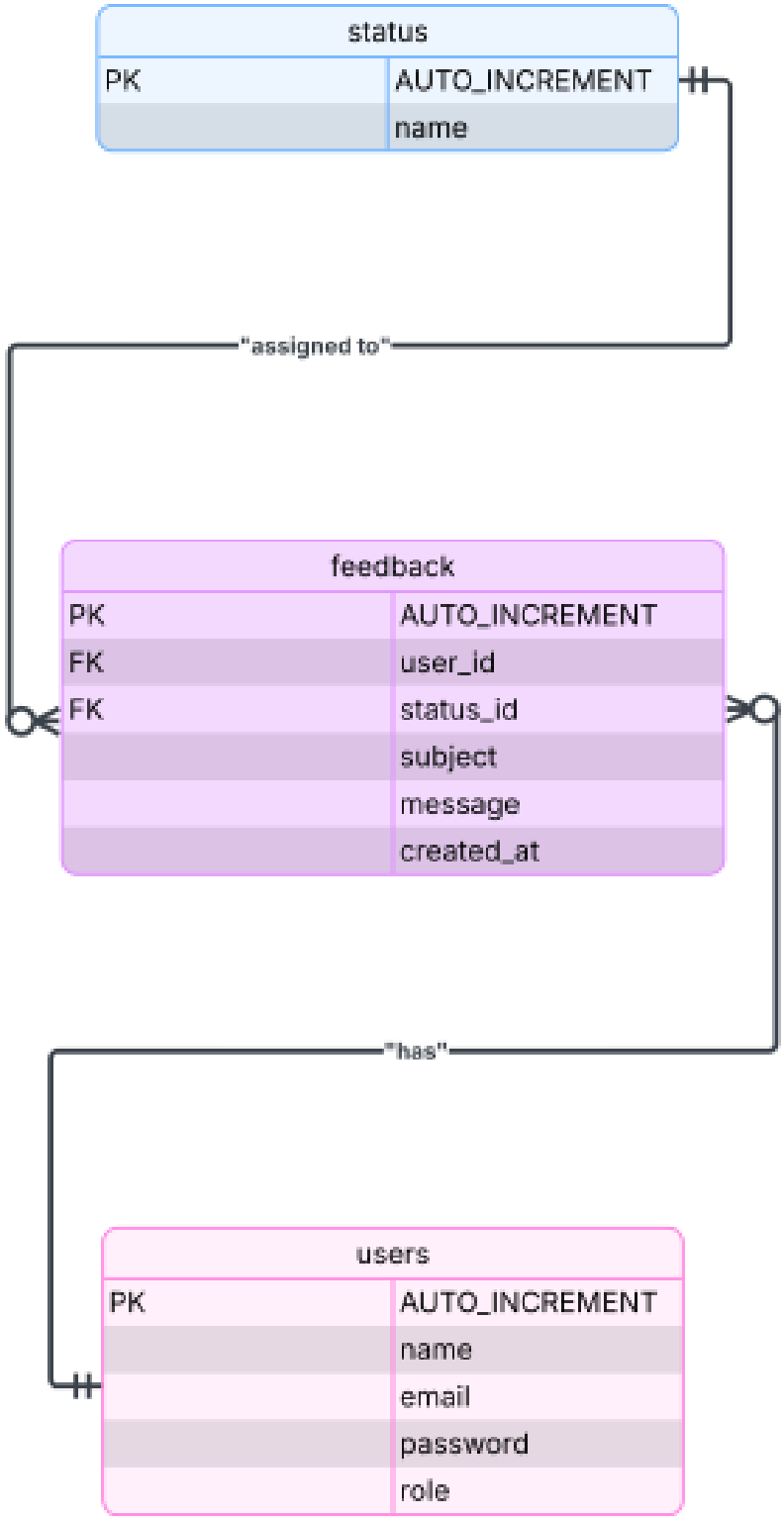


3. **File/folder structure** - Planned organization of your project files

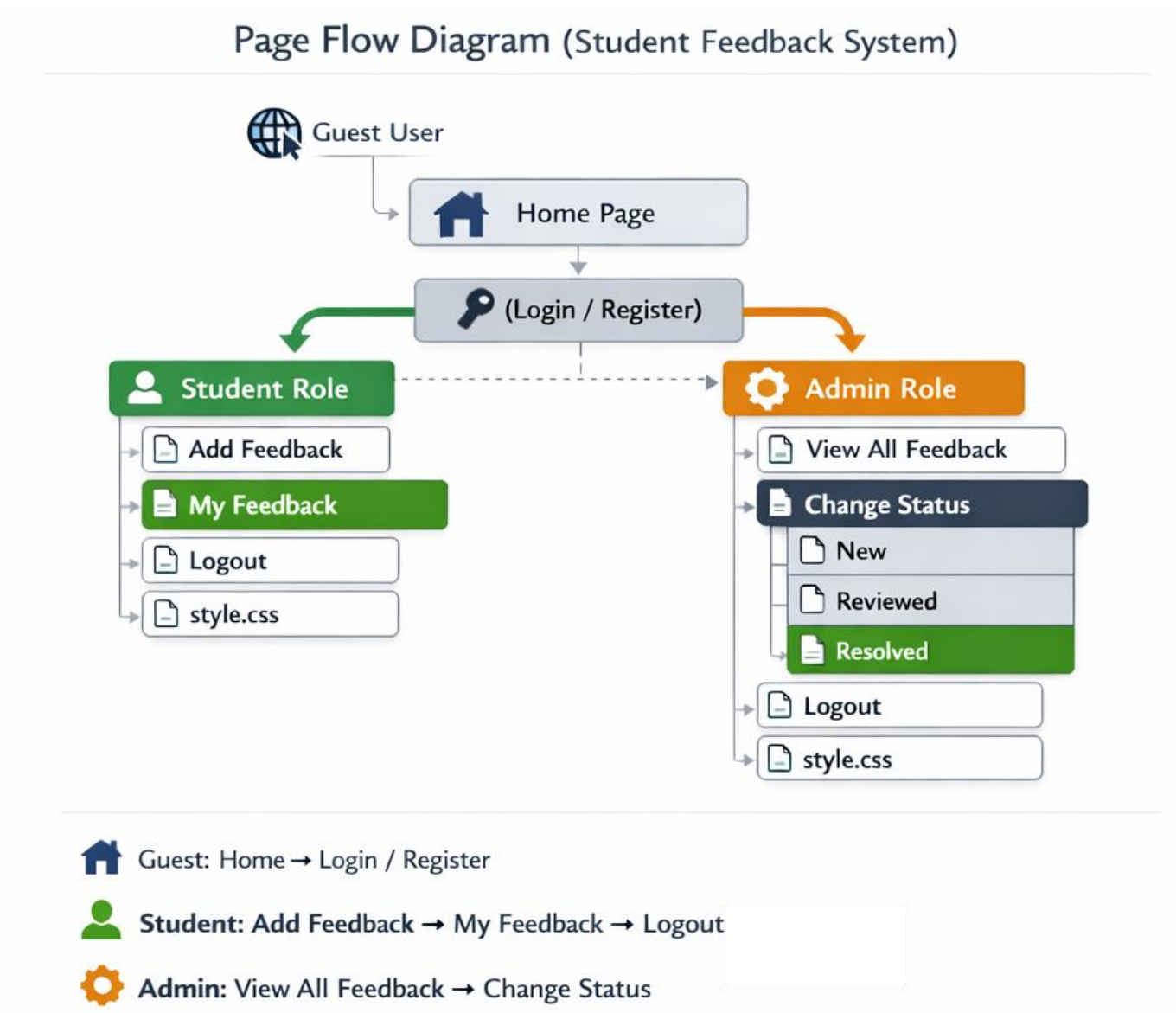


4. **Database design (ER diagram)** - All tables, columns, relationships, keys

This diagram shows the database tables and their relationships.



5. **Page flow diagram** - Navigation paths for different user roles



This diagram shows the navigation flow for both students and administrators in the Student Feedback System. The student can register, log in, submit feedback, view their submitted feedback, and track the status of each feedback. The admin can log in, view all submitted feedback, update feedback status (New, Reviewed, Resolved), delete feedback if necessary, and log out.

Features List

1. User Access

- **1.1 Login Page:** A simple and secure login form that allows students and admins to access their accounts using email and password.
- **1.2 Logout Function:** Allows users to securely log out and end their session to protect their account.

2. Feedback Management (Student Side)

- **2.1 Add Feedback Form:** A form that enables users to submit feedback including subject and message.
- **2.2 View My Feedback:** A page where users can view all the feedback they have submitted.
- **2.3 Feedback Details:** Displays detailed information about each feedback such as subject, message, status, and date.
- **2.4 Feedback Status Tracking:** Allows users to track the status of their feedback (New / Reviewed / Resolved).

3. System Interaction

- **3.1 Form Validation:** Input validation using JavaScript to ensure all required fields are correctly filled.
- **3.2 Responsive Design:** The system is designed to work smoothly on both mobile and desktop devices.

4. Administrator Tools

- **4.1 View All Feedback:** An admin dashboard that displays all feedback submitted by users.
- **4.2 Manage Feedback Status:** Allows admins to update feedback status (New → Reviewed → Resolved).
- **4.3 Feedback Monitoring:** Enables admins to track and organize feedback based on status and date.

5. Data Management

- **5.1 Store Feedback Data:** Saves feedback information in the database.
- **5.2 User Management:** Stores user data (students and admins).
- **5.3 Status Management:** Handles feedback status and links it to each feedback entry.

User Stories

ID	User Story Description
Story 1	As a student, I want to log in, so that I can securely access my account and system features.
Story 2	As a student, I want to submit feedback, so that I can report issues or share my opinions with the administration.
Story 3	As a student, I want to view my submitted feedback, so that I can keep track of what I have reported.
Story 4	As a student, I want to check the status of my feedback, so that I know if it has been reviewed or resolved.
Story 5	As an admin, I want to view all feedback, so that I can monitor student issues and concerns.
Story 6	As an admin, I want to update feedback status, so that I can reflect progress (New, Reviewed, Resolved).
Story 7	As a student, I want the system to validate my input, so that I can avoid submitting incorrect or incomplete data.
Story 8	As a student, I want to use the system on mobile and desktop, so that I can access it anytime and anywhere.

PHASE 2: Frontend Development

1. **HTML pages** (minimum 5 pages) with semantic elements
2. **CSS stylesheet** - Responsive design (must work on mobile)
3. **JavaScript functionality** - Form validation, dynamic content
4. **Screenshots** - Desktop AND mobile views of each page

Overview: In this phase, we developed the frontend of the **Student Feedback System** using HTML5, CSS3, and vanilla JavaScript. The system consists of multiple responsive pages that work on both desktop and mobile devices. No frameworks or libraries were used — all code was written from scratch.

HTML Pages (6ages)

Page	File Name	Description
Home	home.php	Main dashboard page after login with navigation menu
Register	register.php	New user registration form with validation
Login	login.php	User login page using email and password
Add Feedback	add_feedback.php	Form for submitting feedback
My Feedback	my_feedback.php	Displays all feedback submitted by the user
Admin Dashboard	admin_feedback.php	Admin panel to manage all feedback

All pages use **semantic HTML5 elements** to improve structure and accessibility.

Element	Used For
`<header>`	Page header
`<nav>`	Navigation menu
`<main>`	Main content area
`<section>`	Page sections
`<form>`	Input forms
`<table>`	Display feedback data
`<footer>`	Page footer

Login.html:

```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Login</title>
    <link rel="stylesheet" href="style.css">
</head>
<body class="login-page">
<header>
    <h1 class="headerhome">Student Feedback System</h1>
</header>
<main>
    <section class="container">
        <h2>Login</h2>
        <form id="loginForm">
            <input type="email" id="email" placeholder="Email" required>
            <input type="password" id="password" placeholder="Password" required>
            <button type="submit">Login</button>
        </form>
        <p class="error" id="errorMsg"></p>
    </section>
</main>
<footer>
    <p >Student Feedback System @2026</p>
</footer>
<script>
let form = document.getElementById("loginForm");
form.addEventListener("submit", function (e) {
    let email = document.getElementById("email").value;
    let password = document.getElementById("password").value;
    let error = document.getElementById("errorMsg");
    if (email === "" || password === "") {
        e.preventDefault();
        error.textContent = "Please fill all fields.";
    } else {
        error.textContent = "";
        alert("Login successful");
    }
});
</script>
</body>
</html>
```

This is a login page for the Student Feedback System with basic form validation to ensure all fields are filled before submission.

home.html

```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Home</title>
    <link rel="stylesheet" href="style.css">
</head>
<body class="home-page">
<header>
    <h1 class="headerhome">Student Feedback System</h1>
</header>

<main>
    <section class="container">
        <h2 id="p">Welcome</h2>

    <nav class="menu">
    <a href="add_feedback.html">Add Feedback</a>
    <a href="my_feedback.html">My Feedback</a>
    <a href="login.html">Logout</a>
</nav>
    </section>
</main>

<footer>
    <p>Student Feedback System @2026</p>
</footer>

</body>
</html>
```

This is the home page of the Student Feedback System, providing navigation links for adding feedback, viewing submitted feedback, and logging out.

my_feedback.html

```html
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Feedback</title>
    <link rel="stylesheet" href="style.css">
</head>
<body class="myfeedback-page">
<header>
    <h1 class="headerhome">Student Feedback System</h1>
</header>
<main>
    <section class="container">
        <h2>My Feedback</h2>
     <div class="ta">
        <table>
            <tr>
                <th>Subject</th>
                <th>Message</th>
                <th>Status</th>
                <th>Date</th>
            </tr>
            <tr>
                <td>Lab Issue</td>
                <td>The computer is not working.</td>
                <td>Reviewed</td>
                <td>2026-04-14</td>
            </tr>
            <tr>
                <td>Classroom</td>
                <td>The air conditioner is too cold.</td>
                <td>Resolved</td>
                <td>2026-04-13</td>
            </tr>
            <tr>
                <td>Schedule</td>
                <td>The class time is confusing.</td>
                <td>New</td>
                <td>2026-04-12</td>
            </tr>
        </table>
    </div>
            <p><a href="home.html">Back to Home</a></p>
      </section>
  </main>
  <footer>
      <p>Student Feedback System @2026</p>
  </footer>
  </body>
  </html>
```

This page displays the student's submitted feedback in a table, including subject, message, status, and date.

style.css

```
5    body {
6        margin: 0;
7        font-family: Arial, sans-serif;
8        background: rgb(193, 209, 247);
9        color: #222;
10   }
```

Defines the overall design, font, and background of all pages.

```
.login-page main {
    min-height: 60vh;
    display: flex;
    justify-content: center;
    align-items: center;
}
```

Centers the main content vertically and horizontally.

```
68   .container {
69       width: 92%;
70       margin: 20px auto;
71       background-color: #ffffff;
72       padding: 20px;
73       border-radius: 16px;
74       box-shadow: 0 8px 20px rgba(0, 0, 0, 0.12);
75   }
```

Creates a card layout for forms and content sections.

```
input,
textarea {
    width: 100%;
    padding: 12px;
    border: 1px solid #cbd5e1;
    border-radius: 10px;
    font-size: 16px;
    background-color: #f8fafc;
    outline: none;
}
```

Styles input fields for consistent user input design.

```
button {
    padding: 12px;
    background-color: #2563eb;
    color: white;
    border: none;
    border-radius: 10px;
    font-size: 16px;
    cursor: pointer;
}
```

Styles all buttons and hover effects.

```
@media (min-width: 768px) {

    h1 {
        font-size: 2.3rem;
        padding-top: 19px;
    }

    .menu {
        flex-direction: row;
        justify-content: center;
        gap: 20px;
    }
```

Makes the layout responsive for larger screens.

```
144    .myfeedback-page table {
145        width: 100%;
146        border-collapse: collapse;
147        margin-top: 20px;
148        background-color: white;
149    }
150
```

Styles tables for better readability in feedback pages.

3. JavaScript Functionality

Form	Validation Rules
Register	Required fields, email format, password confirmation
Login	Required email and password
Add Feedback	Subject and message must not be empty

Dynamic Features:

- Form input validation before submission
- Display error messages for empty fields
- Alert message on successful login (optional)
- Real-time form validation (basic level)

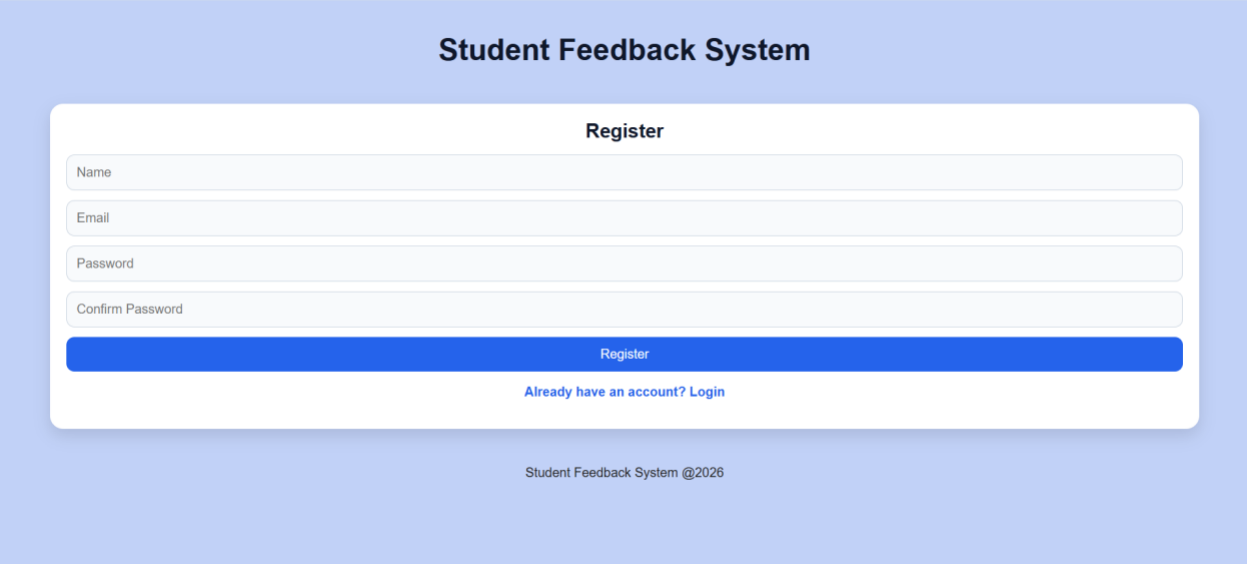
Script:

```
63    <script>
64    let form = document.getElementById("loginForm");
65
66    form.addEventListener("submit", function (e) {
67        let email = document.getElementById("email").value;
68        let password = document.getElementById("password").value;
69        let error = document.getElementById("errorMsg");
70
71        if (email === "" || password === "") {
72            e.preventDefault();
73            error.textContent = "Please fill all fields.";
74        }
75    });
76    </script>
```

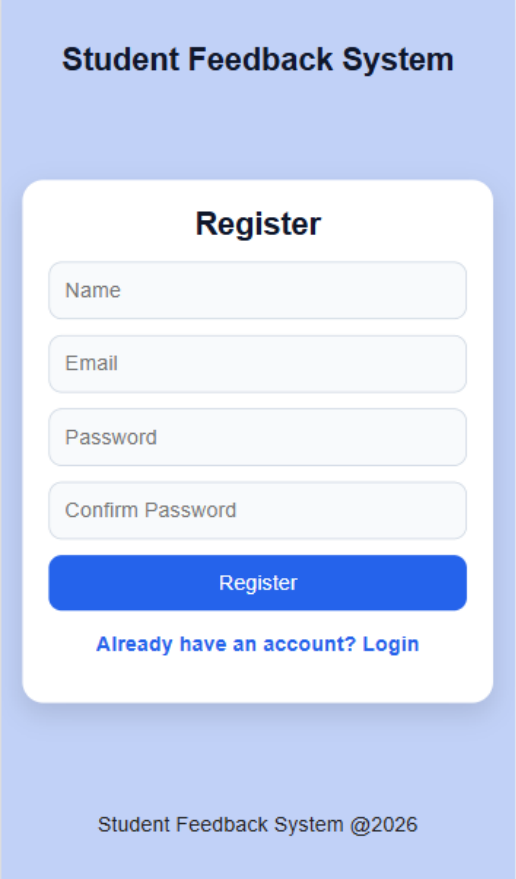
Provides client-side validation to ensure all login fields are filled before submission.

4. Screenshots

Desktop view:



Mobile view:



PHASE 3: Backend & Database

1. **Database** - SQL file to create tables with sample data (minimum 10 records per table)

The system uses a MySQL database named **student_feedback_system1**.

The database includes two main tables:

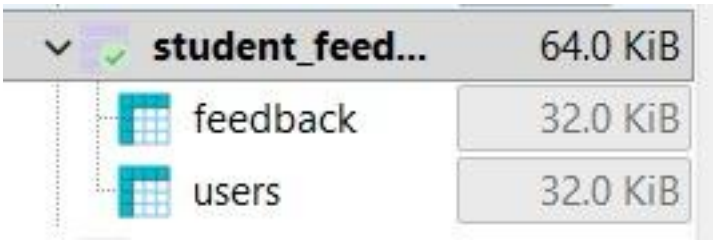
- users
- feedback

Each table contains sample data as required.

Relationships:

- Each feedback entry is linked to a user using user_id (Foreign Key).

student_feedback_system1.sql:



Users table:

#	id	name	email	password	role
1	1	Admin User	admin@yic.edu.sa	$2y$10$lMreBrqZWC3aQD57v.qe0uLptLf6mPFY...	admin
2	2	Sarah Ali	sarah@yic.edu.sa	$2y$10$lMreBrqZWC3aQD57v.qe0uLptLf6mPFY...	student
3	3	Amjad Mohammed	amjad@yic.edu.sa	$2y$10$lMreBrqZWC3aQD57v.qe0uLptLf6mPFY...	student
4	4	Wafa Ahmed	wafa@yic.edu.sa	$2y$10$lMreBrqZWC3aQD57v.qe0uLptLf6mPFY...	student
5	5	Jana Omar	jana@yic.edu.sa	$2y$10$lMreBrqZWC3aQD57v.qe0uLptLf6mPFY...	student
6	6	Shahad Saleh	shahad@yic.edu.sa	$2y$10$lMreBrqZWC3aQD57v.qe0uLptLf6mPFY...	student
7	7	Mona Khaled	mona@yic.edu.sa	$2y$10$lMreBrqZWC3aQD57v.qe0uLptLf6mPFY...	student
8	8	Lama Saad	lama@yic.edu.sa	$2y$10$lMreBrqZWC3aQD57v.qe0uLptLf6mPFY...	student
9	9	Nora Fahad	nora@yic.edu.sa	$2y$10$lMreBrqZWC3aQD57v.qe0uLptLf6mPFY...	student
10	10	Huda Sami	huda@yic.edu.sa	$2y$10$lMreBrqZWC3aQD57v.qe0uLptLf6mPFY...	student
11	11	Sarah	Sar345@gmail.com	$2y$10$0eR6RJB.n1RaHeyQ6mOGh.hCi/KX0XZ...	student
12	12	Rana	rana34@gmail.com	$2y$10$CUEoa9wXluhnRBc77tzAYOJ1c7PKpZkY...	student
13	13	lana	lana123@gmail.com	$2y$10$rJzZj9QeDMkNQTRSmFpHp.cjTXksJupI...	student
14	14	mai	mai123@yic.edu.sa	$2y$10$2fqRIpN8eh3PzhVrV/NPre61Zx3RI/voHj...	student
15	15	dana	dana@yic.edu.sa	$2y$10$4eV3y2eqqHfFn5vs6BxEdu3DdZKHhCW...	student
16	16	joma	joma@yic.edu.sa	$2y$10$xvi0Kh1pK5WeSLXEicoqHeNr/VkO1KRZ...	student
17	17	mela	mela@yic.edu.sa	$2y$10$SG01xjd0F7Vz2wdQaEp/qeaJmhbmbR...	student

Feedback table:

#	id	user_id	subject	message	status	created_at
1	8	9	Cafeteria	Food is expensive	Resolved	2026-04-27 04:50:04
2	9	10	Course	Need more examples	Reviewed	2026-04-27 04:50:04
3	10	2	Exam	Exam time is short	New	2026-04-27 04:50:04
4	12	2	cs360	it is very hard	New	2026-05-17 02:07:15

2. **PHP backend** - CRUD operations using PDO with prepared statements

The backend of the system is built using PHP and PDO with prepared statements to ensure security and prevent SQL injection.

CRUD Operations:

Create:

- Users register accounts
- Students submit feedback

Read:

- Students view their feedback
- Admin views all feedback

Update:

- Admin updates feedback status (New / Reviewed / Resolved)

Delete:

- Admin deletes feedback records

Create:

```
        $stmt = $pdo->prepare(
        "INSERT INTO feedback (user_id, subject, message, status)
        VALUES (:user_id, :subject, :message, 'New')"
    );

    $stmt->execute([
        ':user_id' => $user_id,
        ':subject' => $subject,
        ':message' => $feedback_message
    ]);

        $message = "Feedback submitted successfully!";
    }
}
```

Read:

```
$user_id = $_SESSION["user_id"];
$stmt = $pdo->prepare(
    "SELECT subject, message, status, created_at
    FROM feedback
    WHERE user_id = :user_id
    ORDER BY created_at DESC"
);
```

my_feedback.php

```
$stmt = $pdo->query("
    SELECT feedback.*, users.email
    FROM feedback
    JOIN users ON feedback.user_id = users.id
    ORDER BY feedback.created_at DESC
");
```

admin_feedback.php

Update:

```
18      if ($status == "Reviewed" || $status == "Resolved") {
19          $stmt = $pdo->prepare(
20      "UPDATE feedback SET status = :status WHERE id = :id"
21  );
```

Delete:

```
27  if ($status == "Delete") {
28      $stmt = $pdo->prepare(
29          "DELETE FROM feedback WHERE id = :id"
30      );
```

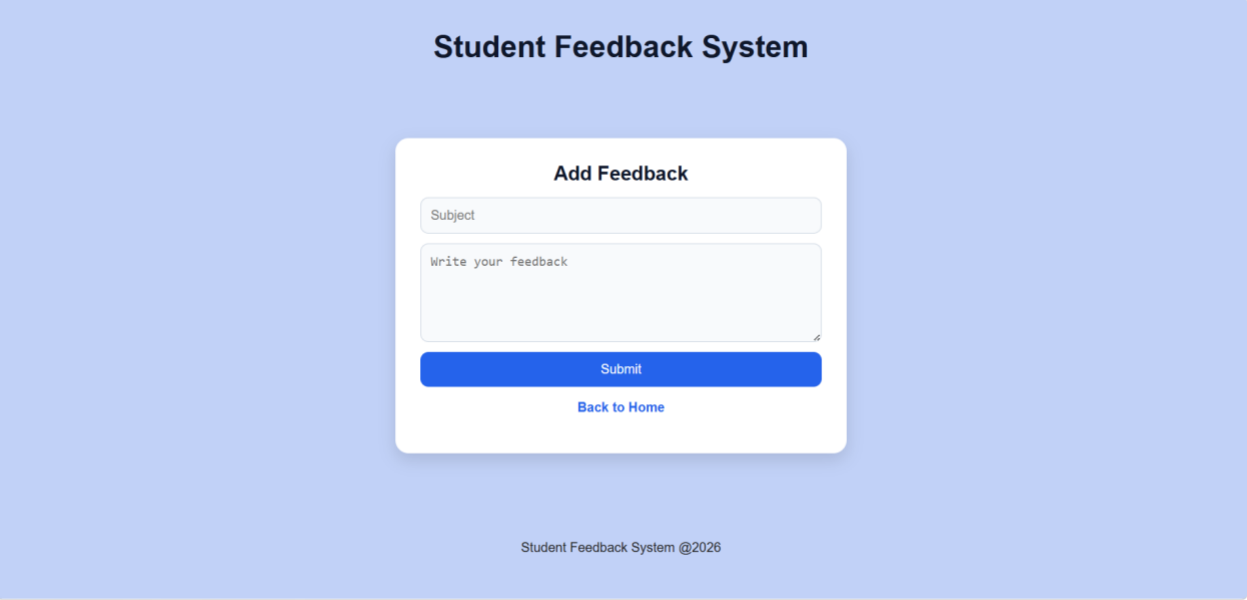


Figure: Create operation - student submits new feedback using Add Feedback page.

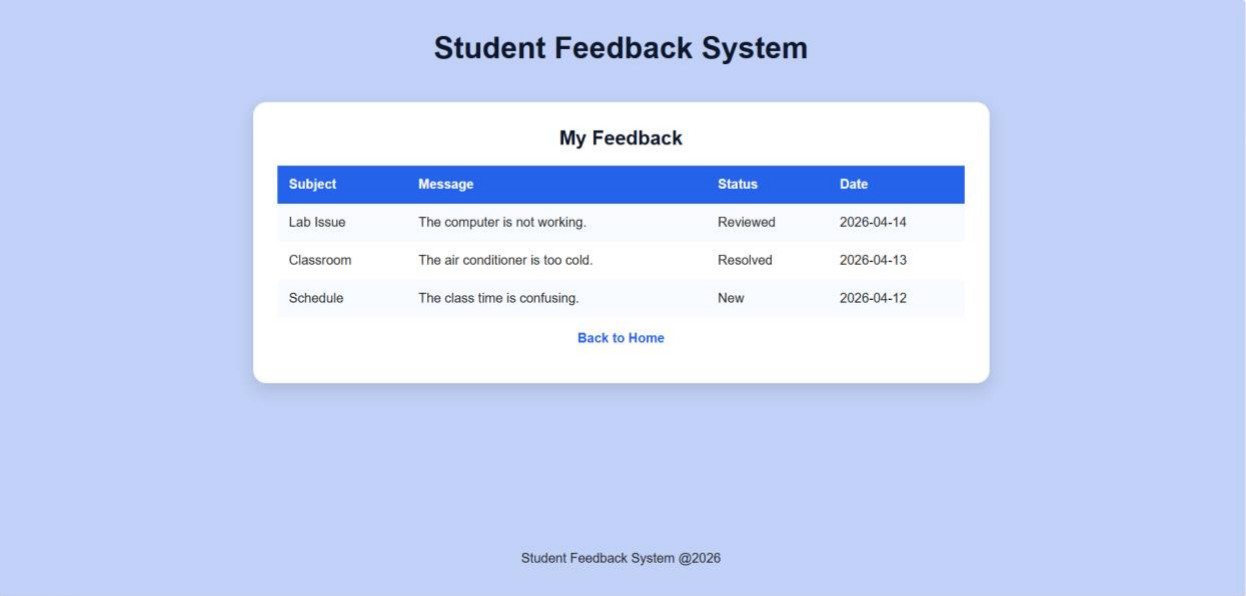


Figure: Read operation - displaying student submitted feedback in My Feedback page.

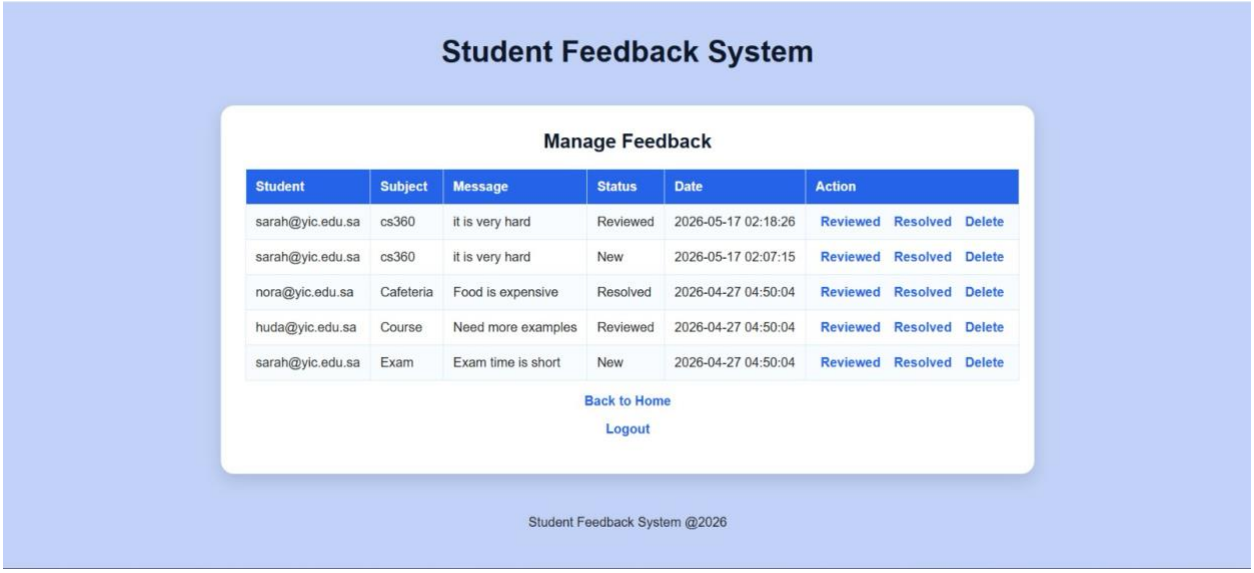


Figure: Read operation - admin views all submitted feedback in Admin Dashboard page.

Figure: Update operation - admin updates feedback status (New, Reviewed, Resolved) in Admin Dashboard page.

Figure: Delete operation - admin removes feedback from the system in Admin Dashboard page.

3. **User authentication** - Registration, login, logout with sessions

The system uses PHP sessions for authentication.

Users can:

- Register
- Login
- Logout securely

Passwords are encrypted using:

- password_hash()
- password_verify()

Sessions are used to protect pages and control access.

Files:

- register.php
- login.php
- logout.php

```
30              $hashedPassword = password_hash($password, PASSWORD_DEFAULT);
31              $stmt = $pdo->prepare(
32          "INSERT INTO users (name, email, password, role)
33          VALUES (:name, :email, :password, :role)"
34      );
35      $stmt->execute([
36          ':name' => $name,
37          ':email' => $email,
38          ':password' => $hashedPassword,
39          ':role' => $role
40      ]);
41              header("Location: login.php");
42              exit();
43          }
```

This code hashes the user password and stores the new user data securely in the database.

```php
    if ($user && password_verify($password, $user["password"])) {
        $_SESSION["user_id"] = $user["id"];
        $_SESSION["user_name"] = $user["name"];
        $_SESSION["user_role"] = $user["role"];

        header("Location: home.php");
        exit();
    } else {
        $message = "Invalid email or password.";
    }
```

This code verifies the password and creates a session for authenticated users.

```php
<?php
session_start();
$_SESSION = [];
session_destroy();
header("Location: login.php");
exit();
?>
```

This code destroys the session and logs the user out securely.

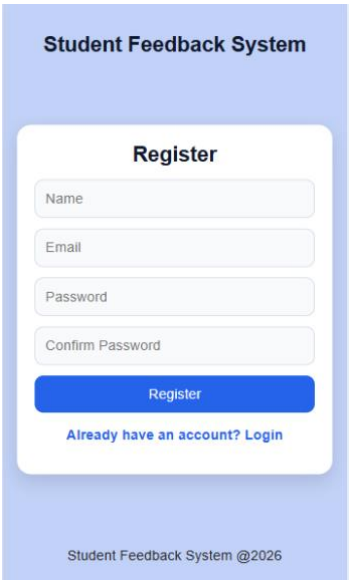


Figure: User registration page

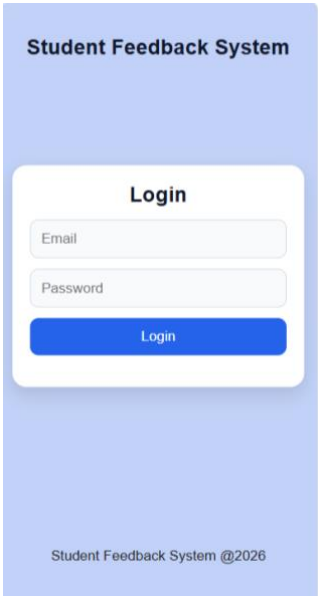


Figure: User login page

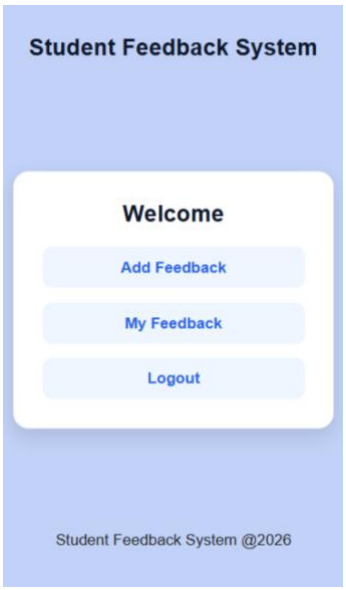


Figure: User logout page

4. **Role-based access** - At least 2 roles (e.g., student and admin)

The system supports two roles:

- Student
- Admin

Student Role:

- Register and login
- Submit feedback
- View own feedback only

Admin Role:

- Login to admin dashboard
- View all feedback
- Update feedback status
- Delete feedback records

Login.php:

```
21          $_SESSION["user_role"] = $user["role"];
```

home.php:

```
29          <?php if ($_SESSION["user_role"] == "student") { ?>
30              <a href="add_feedback.php">Add Feedback</a>
31              <a href="my_feedback.php">My Feedback</a>
32          <?php } ?>
33
34          <?php if ($_SESSION["user_role"] == "admin") { ?>
35              <a href="admin_feedback.php">Manage Feedback</a>
36          <?php } ?>
```

admin_feadback.php:

```
10  if ($_SESSION["user_role"] != "admin") {
11      die("Access denied");
12  }
```

my_feadback.php:

```
10    if ($_SESSION["user_role"] != "student") {
11        die("Access denied");
12    }
```

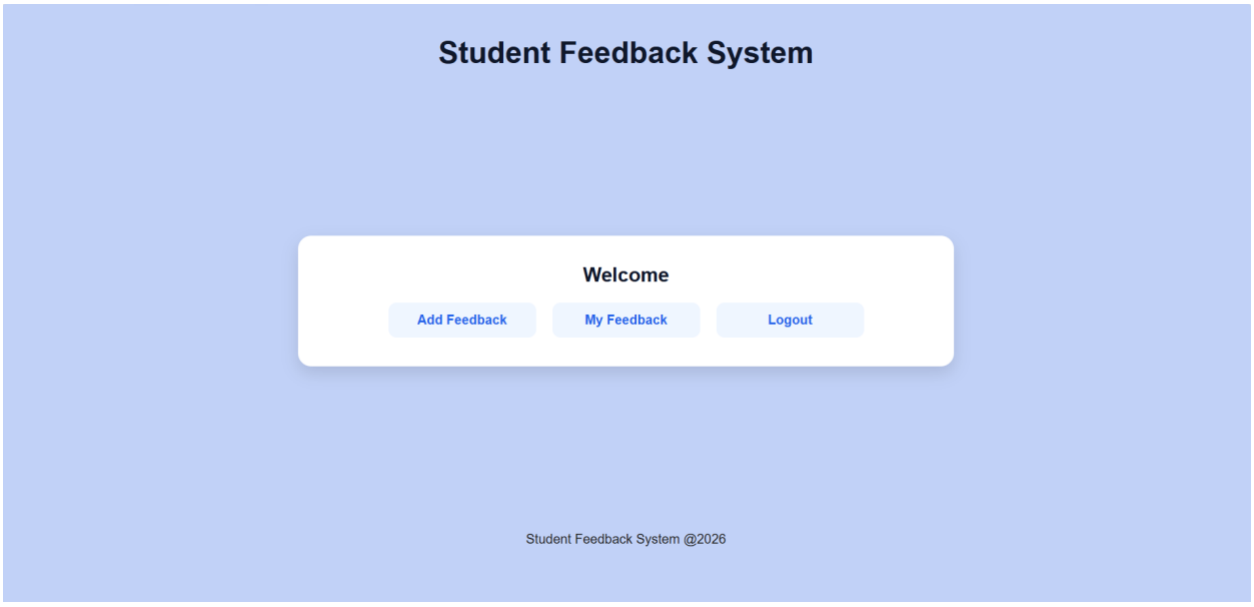


Figure: Student interface showing access to student features only.

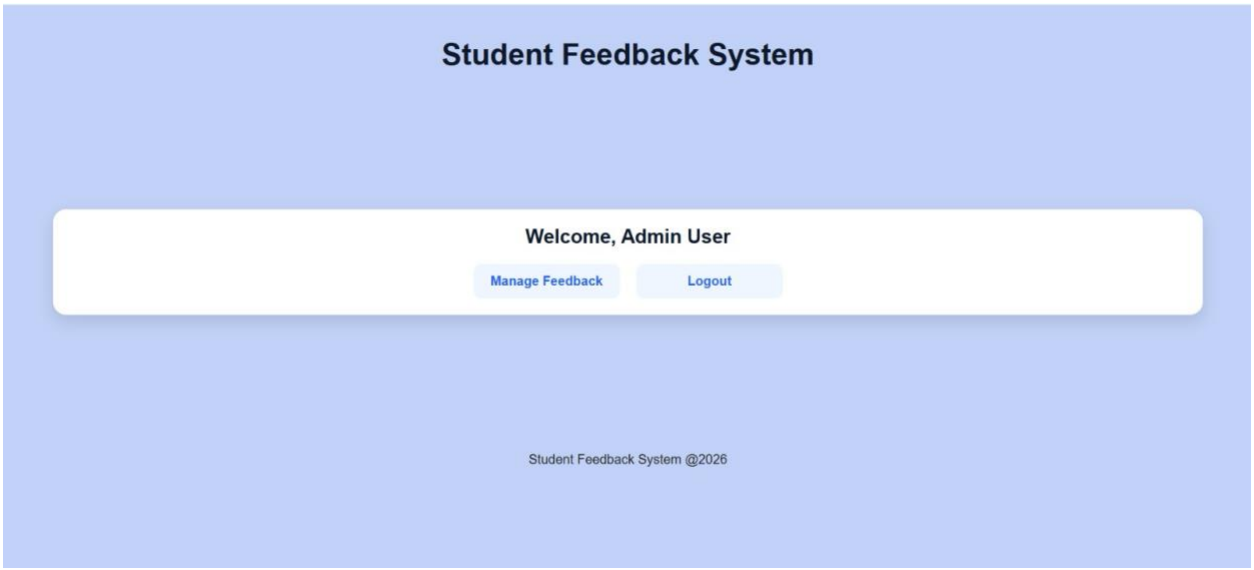


Figure: Admin interface showing access to admin features only.

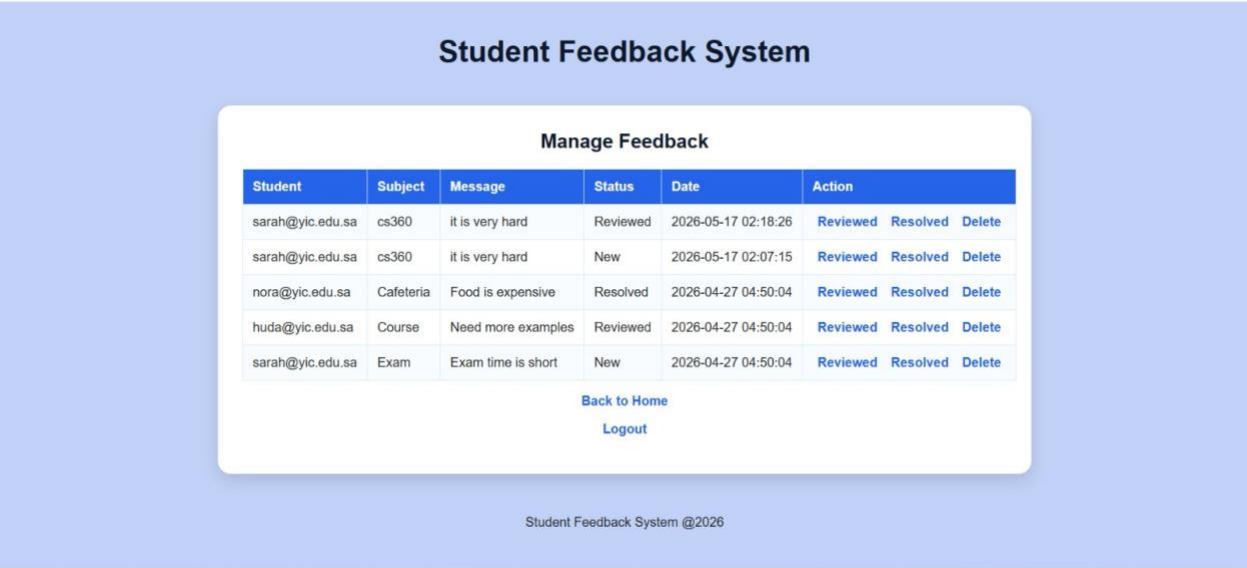


Figure: Admin dashboard with full control to view, update, and delete feedback.

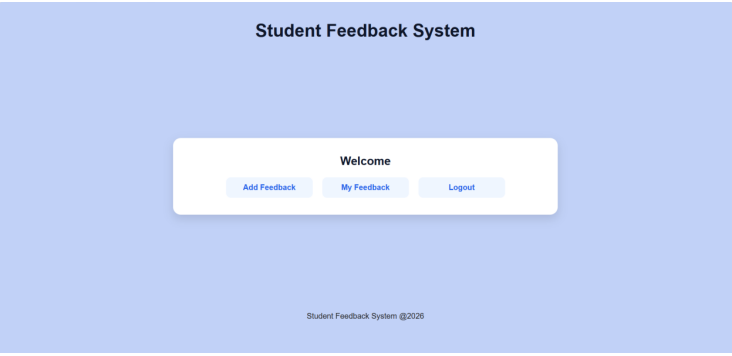
PHASE 4: Integration, Security and Submission

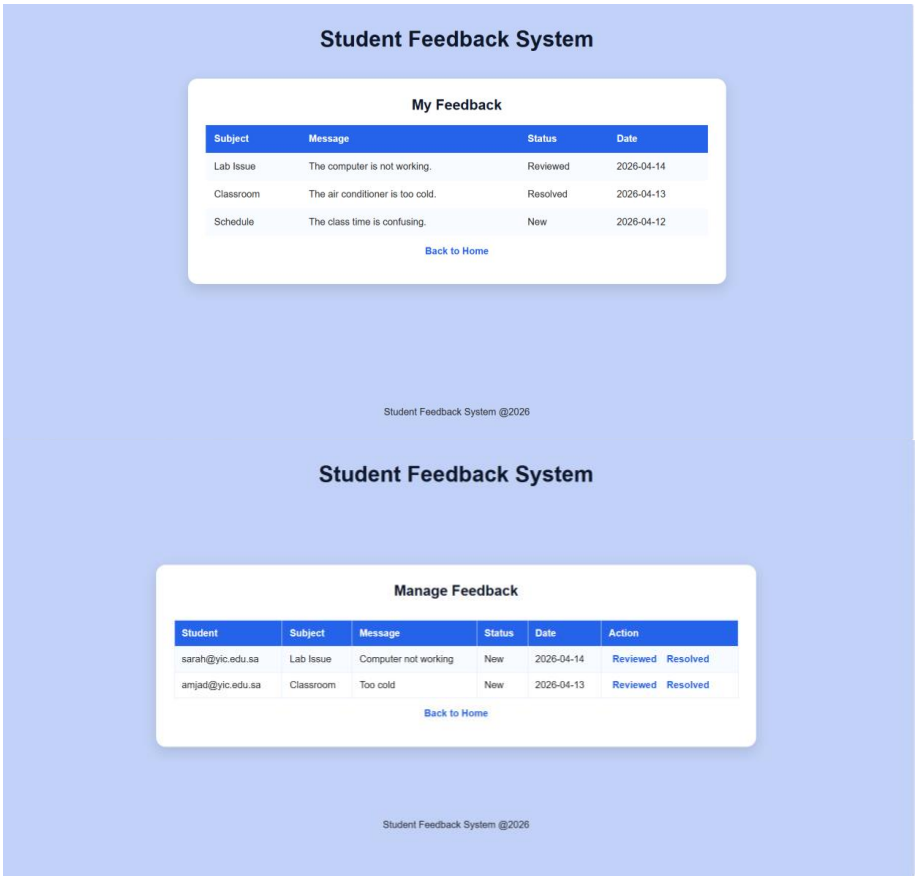
1. **Complete working application** - All features integrated

The Student Feedback System is fully integrated and all features are working together successfully.
The system allows:

- Students to register, log in, submit feedback, and view their feedback.
- Admins to log in, view all feedback, update feedback status, and delete feedback.

All frontend pages are connected with the backend using PHP and the database using PDO. The system ensures smooth navigation between pages based on user role.





The system is fully integrated, allowing students and admins to access all features after login. Navigation changes based on user role.

2. **Security implementation** - Input validation, XSS prevention, CSRF tokens

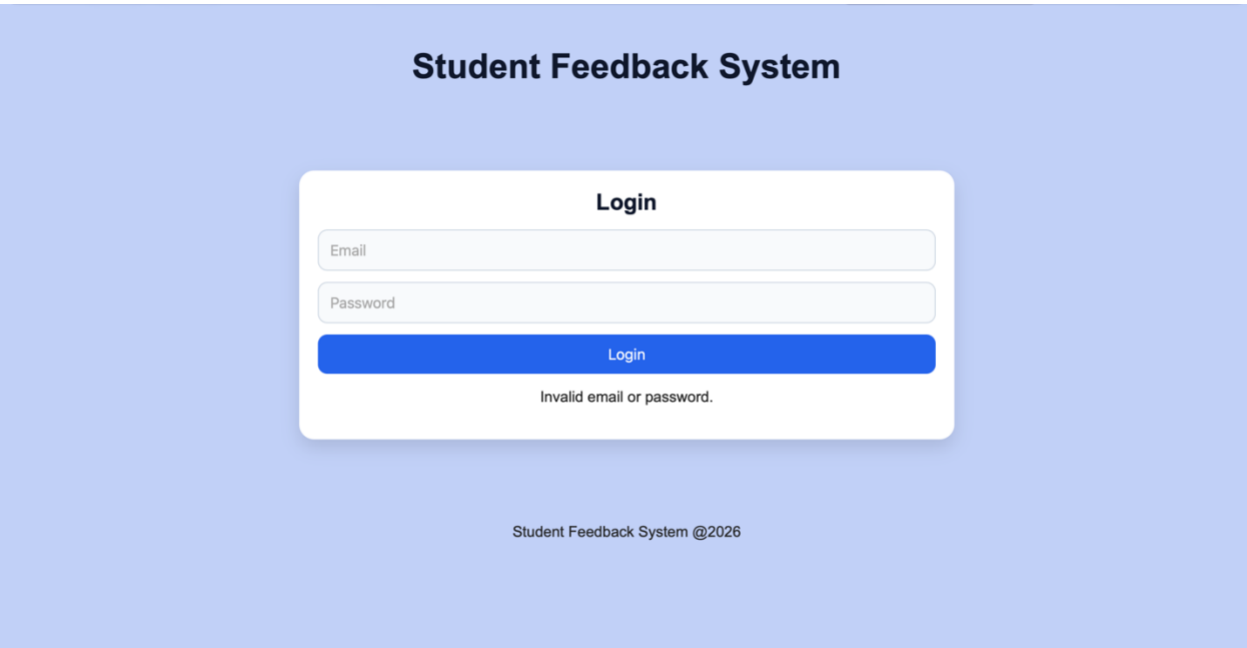
The system includes multiple security techniques to protect user data and prevent attacks:

Input Validation
All forms validate user input before submission to ensure required fields are filled and data is correct.

Password Security
Passwords are encrypted using:

- password_hash()
- password_verify()

This ensures that passwords are not stored in plain text.
SQL Injection Prevention
The system uses PDO prepared statements in all database queries to prevent SQL injection attacks.



The system validates user credentials during login. If the email or password is incorrect, an error message is displayed and access is denied.

```
21          $stmt = $pdo->prepare( "SELECT id FROM users WHERE email = :email"
22      );
23      $stmt->execute([
24          ':email' => $email
25      ]);
```

Session Management

PHP sessions are used to:

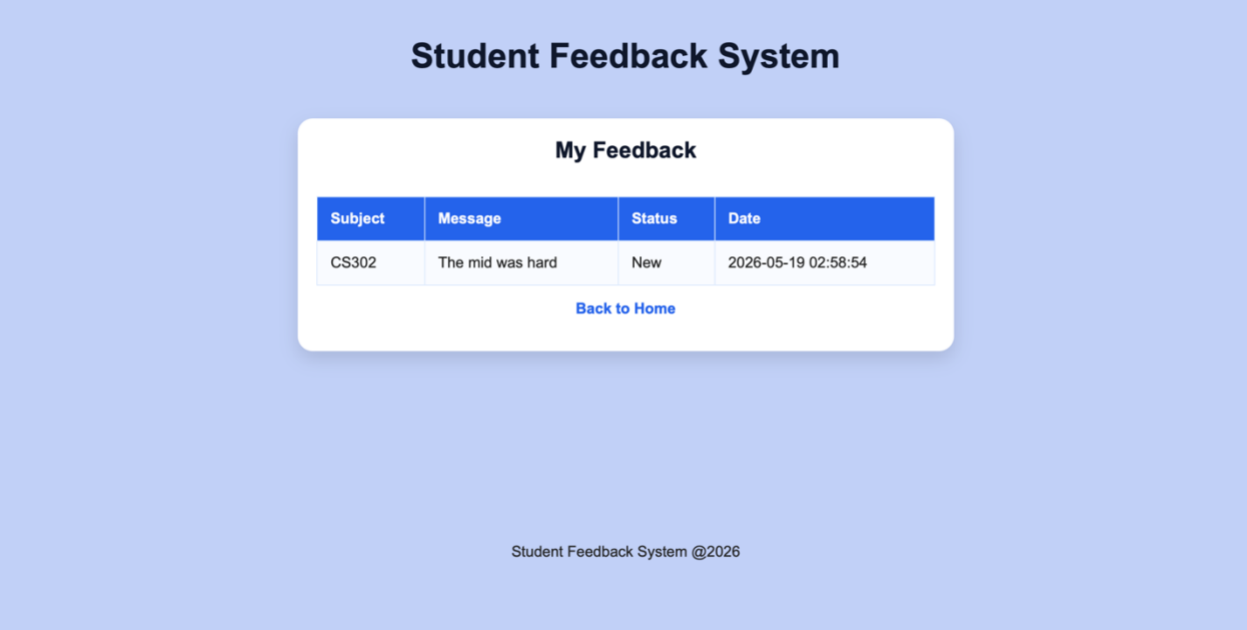
- Keep users logged in
- Restrict access to pages
- Store user role (student/admin)

XSS Protection

User input is sanitized using:

```
76          <p class="success"><?php echo htmlspecialchars($message); ?></p>
```

This prevents malicious scripts from being executed in the browser.



User input is sanitized using htmlspecialchars() to prevent Cross-Site Scripting (XSS) attacks.

CSRF Protection

A CSRF token is generated and validated in forms:

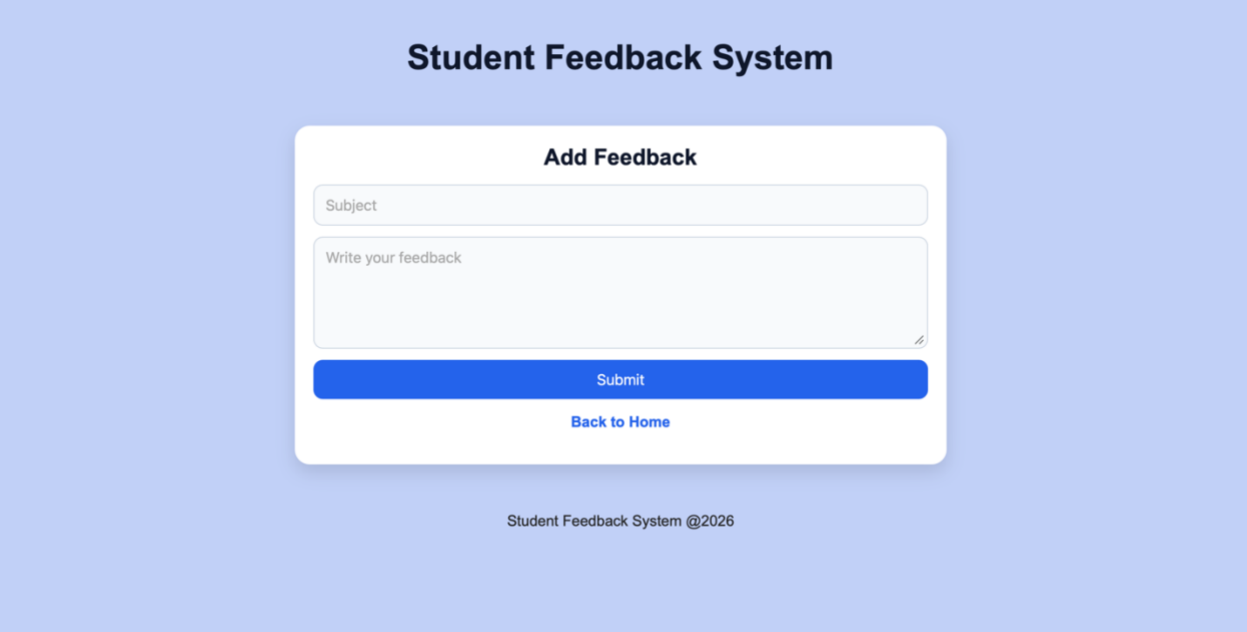
```php
$message = "";
if (!isset($_SESSION["token"])) {

    $_SESSION["token"] =
        bin2hex(random_bytes(32));
}
```

```php
    if (
    !isset($_POST["token"]) ||
    $_POST["token"] !== $_SESSION["token"]
) {

    die("Invalid CSRF Token");
}
```

This prevents unauthorized form submissions.



A CSRF token is included in the feedback form to protect the system from unauthorized form submissions.

3. **Video walkthrough** (3-5 minutes) - YOU demonstrating YOUR application

https://drive.google.com/file/d/1a1cbKBfEXF5m0ABoA2k89X5DizpBQsGV/view?usp=share_link

https://github.com/sarahmg1424-hash/project-CS381

4. **README file** - Setup instructions, login credentials, feature list

Setup Instructions

1. Start Laragon/XAMPP
2. Import database
3. Place project in www or htdocs
4. Open: http://localhost:8080/projects381/login.php

Login Credentials

Admin:

- admin@gmail.com / 12345678

Student:

- sara@gmail.com / 12345678

Features

- Register
- Login / Logout
- Add Feedback
- View Feedback

- Admin Manage Feedback
- Update & Delete Feedback

Submission & Defense

Deliverables:

- Complete, polished application
- Final database seeder
- Clean code (no debug statements, proper naming)
- Final report

Final Defense (10-15 minutes per student):

- Full application walkthrough, logging in as each role
- Instructor asks 5-7 questions about your code and design decisions
- 1 live modification challenge (more complex tasks)
- Explain one part of your code that was hardest to implement and why

Project Report (All items should be in your project report)

- Title Page (page 1 and 2 – Fill up your names and IDs)
- Problem Description (chosen Service Module)
- Table of Contents
- Description of System
- Updated ER diagram
- System Output
- AI disclosure – discussion on how you have used AI to assist your development
- Challenges
- Future improvement
- Conclusion

Submission Guidelines

- All code including the final report and presentation slide pushed to a **private/public GitHub/GitLab repository** shared with the instructor

No Frameworks Policy

YOU MUST WRITE ALL CODE FROM SCRATCH – NO FRAMEWORKS OR LIBRARIES ALLOWED

NOT ALLOWED	ALLOWED
Bootstrap, Tailwind, Foundation jQuery, React, Vue, Angular PHP frameworks (Laravel, CodeIgniter) CSS preprocessors (Sass, Less) Template engines (Blade, Twig) Any npm/composer packages	Vanilla HTML5, CSS3, JavaScript (ES6) Plain PHP with PDO Google Fonts for typography Font Awesome for icons only Your own reusable CSS/JS files

Reason: The goal is to learn fundamentals. Using frameworks hides the core concepts you need to understand. You will learn frameworks in advanced courses.

Project Evaluation Rubric

Grading Rubric

Component	CLO	Marks	Excellent (100%)	Good (75%)	Needs Work (50%)	Poor (25%)
Architecture Design	3.1	20	Complete system diagram, clear file structure, detailed ER with relationships, logical page flow	All diagrams present, minor issues	Some diagrams missing or unclear	Poor or no architecture planning
Phase 1: Planning	3.1	10	Clear wireframes, complete user stories	Most components present	Basic planning	Incomplete
Phase 2: Frontend	1.3	10	Semantic HTML, responsive CSS (no frameworks), working vanilla JS	Good HTML/CSS, basic JS	Works but not semantic	Broken or uses frameworks
Phase 3: Backend	1.3	10	Full CRUD, auth, proper PDO, follows planned architecture	CRUD works, basic auth	Partial functionality	Major issues
Phase 4: Integration & Security	2.1	10	Everything works, secure, matches architecture	Most features work	Some features broken	Not integrated
Code Quality	2.1	10	Clean, organized, commented, follows file structure	Mostly organized	Messy but works	Disorganized, hard to read
Video Walkthrough	3.2	10	Clear demo, face shown, all features	Good demo, minor issues	Basic demo	Missing/poor quality
Project Defense (Viva)	3.2	20	Explains code AND	Explains most code/design	Struggles with explanations	Cannot explain own code

			architecture confidently			
	TOTAL		**100 marks**			